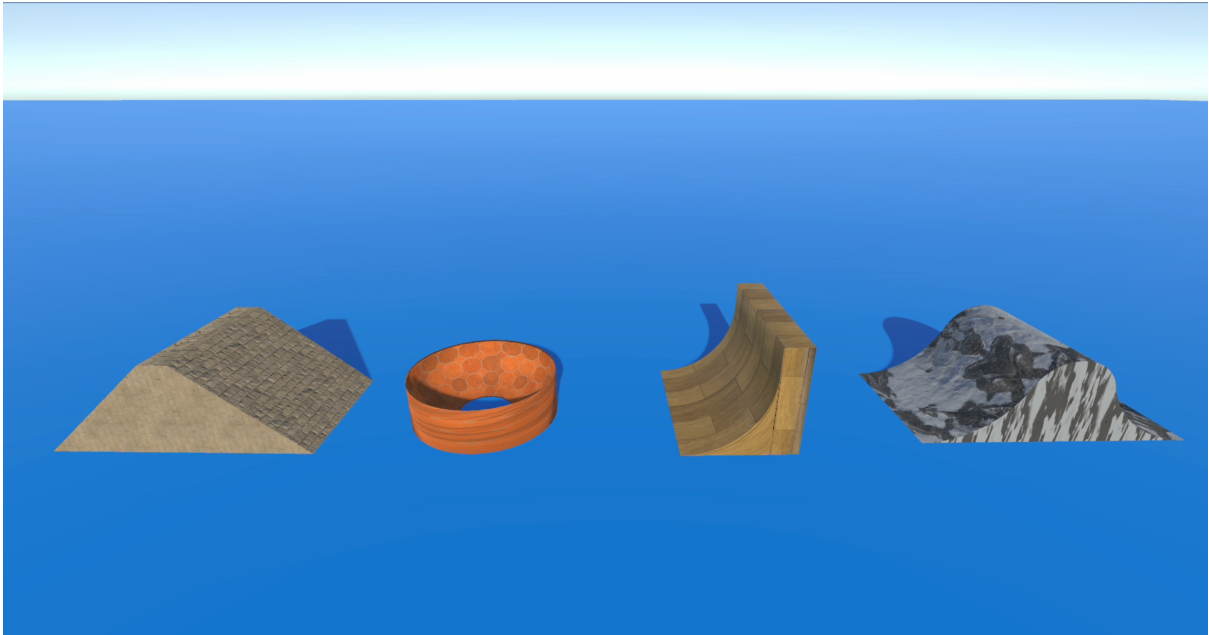


Ramp Builder Documentation



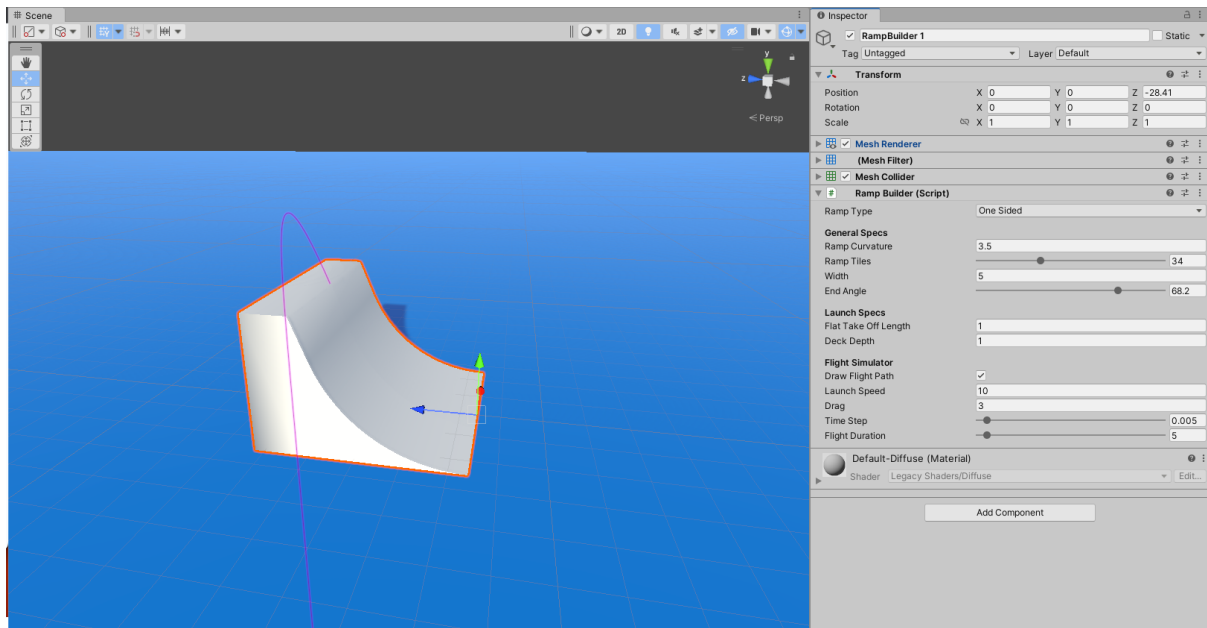
[Setup Guide](#)

[Procedural Generation](#)

Setup Guide

This package includes four types of ramps: one-sided, two-sided, tabletop and curved ramps.

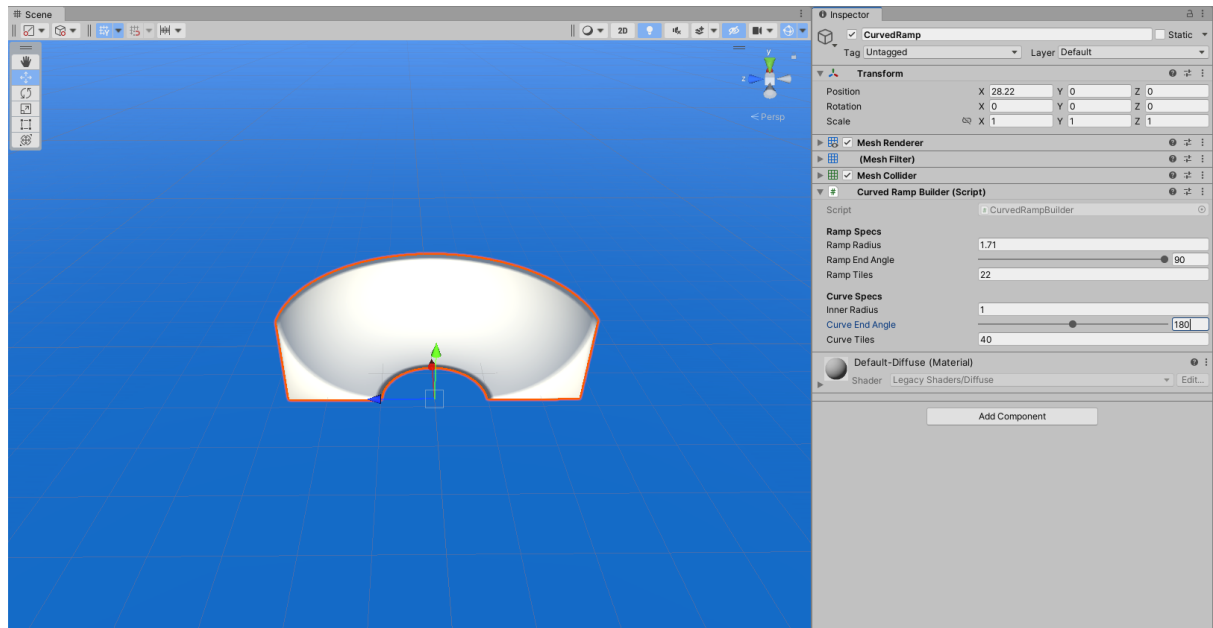
The first three types are created with the RampBuilder.cs script. Simply add the RampBuilder script to any GameObject to turn it into a ramp mesh.



The editor will give you various ways to customise your ramp. In the Ramp Type you can select: one-sided, two-sided or tabletop.

The other settings depend on the selected type but they all include things like the curvature radius, width, take-off angle and ramp tiles (which is related to the number of polygons used).

If you want to create a curved ramp you'll have to include a different script: CurvedRampBuilder.cs.



I've made a YouTube video that shows how each parameter affects the ramp, watch it [here](#).

Procedural Generation

Creating the ramps and changing their mesh at runtime can be done with just a few extra steps.

Example:

Let's say we want to animate the *endAngle* of a ramp from 10° to 90° and back with an interval of 1 second.

We first go to *RampBuilder.cs* and we change the *endAngle* property to have public access.

```
public class RampBuilder : MonoBehaviour
{
    4 references
    [SerializeField] RampType rampType;
    8 references
    [SerializeField] SegmentData[] segments;

    [Header("General Specs")]
    25 references
    [SerializeField] float rampCurvature = 1;
    [Range(1, 100)]

    16 references
    [SerializeField] int rampTiles = 20;
    13 references
    [SerializeField] float width = 1;
    [Range(0.1f, 90)]

    34 references
    [SerializeField] public float endAngle = 30;
```

Then let's create a new script *RampAngleAnimation.cs*:

```
using UnityEngine;
using RampCreation;

public class RampAngleAnimation : MonoBehaviour
{
    RampBuilder rampBuilder;

    void Start ()
    {
        rampBuilder = GetComponent<RampBuilder>();
    }

    void Update()
    {
        // Animate the ramp angle from 10° to 90° and back over 1 second
        float angle = Mathf.PingPong(Time.time, 1) * 80 + 10;
        rampBuilder.endAngle = angle;
        rampBuilder.BuildRampSegments();
    }
}
```

We can add this script to any GameObject that has the *RampBuilder.cs* component and now its endAngle will animate from 10° to 90°.